"*Platform Engineering & DevOps*" Trend Report is now LIVE! Learn how internal platforms help developers ship faster with less friction

Download New Trend Report

(https://dzone.com/link/2026-tr-platform-eng-devops-announcement)

Live Webinar: Learn how malicious AI skills compromise developer workflows and what defenses work now.

Register Free Today

(https://cvent.me/eKDokG?utm_source=Announcements&utm_medium=DzoneWeb&utm_campaign=Snyk-2)

AI-ready data starts with modernization. Join DZone + Informatica live on 6/16 at 1 PM ET.

Register for Webinar

(https://cvent.me/AVwYLb?utm_source=Announcements&utm_medium=DzoneWeb&utm

RELATED

From Command Lines to Intent Interfaces: Reframing Git Workflows Using Model Context Protocol (/articles/from-command-lines-to-intent-interfaces-reframing-git)

Secrets in Code: Understanding Secret Detection and Its Blind Spots (/articles/secrets-in-code-detection-and-its-blind-spots)

How to Push Docker Images to AWS Elastic Container Repository Using GitHub Actions (/articles/push-docker-images-to-aws-ecr-using-github-actions)

Optimizing GitHub Access Management for Enterprises: Enhancing Security, Scalability, and Continuity with Jenkins GitHub App Authentication and Load Balancing (/articles/optimizing-github-access-management-for-enterprises)

✕

✕

(/) REFCARDS (/REFCARDZ) TREND REPORTS (/TREND-REPORTS) EVENTS (/EVENTS) (/users/login.html)

DZone (https://dzone.com) / Testing, Deployment, and Maintenance (https://dzone.com/testing-deployment-and-maintenance) / Deployment (https://dzone.com/deployment) / Complete Guide: Managing Multiple GitHub Accounts on One System

Culture and Methodologies (/culture-and-methodologies) Data Engineering (/data-engineering) Software Design and Architecture (/software-design-and-architecture) Coding (/coding) Testing, Deployment, and Maintenance (/testing-deployment-and-maintenance) Partner Zones

Complete Guide: Managing Multiple GitHub Accounts on One System

This guide helps you configure and manage multiple GitHub accounts on the same system (e.g., personal and work accounts).

By Prathap Reddy M (/users/5191102/prathap532.html) · Jun. 19, 25 · Tutorial

Likes (4) Comment (0) Save Tweet Share 3.0K Views

This comprehensive guide helps developers configure and manage multiple GitHub accounts on the same system, enabling seamless switching between personal, work, and client accounts without authentication conflicts.

Overview

Managing multiple GitHub accounts on a single development machine is a common requirement for modern developers. Whether you're maintaining separate personal and professional identities, working with multiple clients, or contributing to different organizations, proper account management ensures secure access control and maintains clear commit attribution.

This guide provides three distinct approaches to handle multiple GitHub accounts (https://dzone.com/articles/mastering-multiple-github-accounts), from basic manual switching to advanced automated configuration. Each method has its own advantages and is suitable for different workflow requirements.

Key Benefits:

- Secure isolation between different GitHub accounts
- Automatic switching based on project directory
- Prevention of accidental commits to wrong repositories
- Streamlined authentication across multiple accounts
- Clear audit trail for commit attribution

Understanding the Challenge

When working with multiple GitHub accounts, developers face several technical challenges:

Authentication Conflicts: Git and SSH typically use default credentials, leading to access denials when switching between accounts. The system may authenticate with the wrong account, resulting in permission errors.

Identity Management: Each commit should be attributed to the correct developer identity. Using the wrong name or email in commits can cause confusion in project history and violate organizational policies.

Repository Access: Different repositories may belong to different accounts or organizations, requiring specific authentication credentials for each context.

Workflow Complexity: Manual switching between accounts is error-prone and time-consuming, especially when working on multiple projects simultaneously.

Security Concerns: Improper configuration can lead to credential leakage (https://dzone.com/articles/what-to-do-if-you-expose-a-secret-how-to-stay-calm) or unauthorized access to repositories.

Prerequisites

Before implementing any of the solutions in this guide, ensure you have:

- **Git installed** (version 2.20 or later recommended)
- **SSH** (https://dzone.com/articles/ssh-tutorial-nice-amp-easy-video) **client** available on your system
- **Multiple GitHub accounts** with appropriate access permissions
- **Command line access** with ability to edit configuration files
- **Basic understanding** of Git workflows and SSH key concepts
- **Administrative rights** to modify SSH and Git configurations

ABOUT US
About DZone (/pages/about)
Support and feedback
Community research (/pages/dzone-community-research)

ADVERTISE
Advertise with DZone (https://advertise.dzone.com)

CONTRIBUTE ON DZONE
Article Submission Guidelines (/articles/dzones-article-submission-guidelines)
Become a Contributor (/pages/contribute)
Core Program (/pages/core)
Visit the Writers' Zone (/writers-zone)

LEGAL
Terms of Service (https://technologyadvice.com/terms-conditions/)
Privacy Policy (https://technologyadvice.com/privacy-policy/)

CONTACT US
3343 Perimeter Hill Drive
Suite 215
Nashville, TN 37211
support@dzone.com (mailto:support@dzone.com)

Let's be friends:

Step-by-Step Setup

1. Generate SSH Keys for Each Account

Shell

```
# For your primary account (if you don't have one already)
ssh-keygen -t ed25519 -C "primary-email@example.com" -f ~/.ssh/id_ed25519
# For your secondary account
ssh-keygen -t ed25519 -C "secondary-email@example.com" -f ~/.ssh/id_ed25519_secondary
```

Note: When prompted for a passphrase, you can either:

- Leave it empty (press Enter) for convenience
- Set a passphrase for additional security

2. Add SSH Keys to GitHub Accounts

Copy your public keys:

Shell

```
# Primary account public key
cat ~/.ssh/id_ed25519.pub
# Secondary account public key
cat ~/.ssh/id_ed25519_secondary.pub
```

Add to GitHub:

1. Log into each GitHub account
2. Go to Settings → SSH and GPG keys
3. Click New SSH key
4. Paste the respective public key
5. Give it a descriptive title (e.g., "Work Laptop - Primary Account")

3. Configure SSH Config File

Create or edit `~/.ssh/config`

Shell

```
# Edit the SSH config file
nano ~/.ssh/config
```

Add the following configuration:

Shell

```
# Primary GitHub account (default)
Host github.com
HostName github.com
User git
IdentityFile ~/.ssh/id_ed25519
```

```
# Secondary GitHub account
Host github-secondary
HostName github.com
User git
IdentityFile ~/.ssh/id_ed25519_secondary
```

Important:

- The User must always be ***git*** for GitHub
- Replace `github-secondary` with any alias you prefer (e.g. `github-work,github-personal`)
- Do NOT add ***.com*** to your custom host aliases

4. Set Proper Permissions

Shell

```
chmod 600 ~/.ssh/config
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519*
```

5. Test SSH Connections

Shell

```
# Test primary account
ssh -T git@github.com
# Test secondary account
ssh -T git@github-secondary
```

Expected output:

Hi username! You've successfully authenticated, but GitHub does not provide shell access.

Usage Examples

Cloning Repositories

Shell

```
# Clone with primary account (default)
git clone git@github.com:username/repo-name.git
# Clone with secondary account
git clone git@github-secondary:username/repo-name.git
```

Setting Up Existing Repositories

Shell

```
# Check current remote
git remote -v
# Switch to secondary account
git remote set-url origin git@github-secondary:username/repo-name.git
# Verify the change
git remote -v
```

Configure Git Identity Per Repository

Shell

```
# Set identity for current repository
git config user.name "Your Work Name"
git config user.email "work-email@company.com"
# Or for personal projects
git config user.name "Your Personal Name"
git config user.email "personal@gmail.com"
```

Troubleshooting

Common Issues and Solutions

1. "Could not resolve hostname github-secondary"

Problem: Using ***.com*** with your SSH alias or SSH config not properly set up.

Solution:

- Use `git@github-secondary` NOT `git@github-secondary.com`
- Verify your SSH config file exists and has correct permissions

2. "Permission denied (publickey)"

Problem: SSH key not added to GitHub or wrong key being used.

Solutions:

Shell

```
# Test specific key
ssh -i ~/.ssh/id_ed25519_secondary -T git@github.com
# Add key to SSH agent
ssh-add ~/.ssh/id_ed25519_secondary
# Verify key is added to GitHub account
```

3. "Push rejected" or "Access denied"

Problem: Repository remote URL doesn't match your SSH configuration.

Solution:

Shell

```
# Check remote URL
git remote -v
# Update remote URL to use correct SSH alias
git remote set-url origin git@github-secondary:username/repo.git
```

4. Wrong author in commits

Problem: Git using wrong user identity.

Solution:

Shell

```
# Check current identity
git config user.name
git config user.email
# Set correct identity for this repository
git config user.name
"Correct Name"
git config user.email
"correct@email.com"
```

Debug Commands

Shell

```
# Test SSH connection with verbose output
ssh -vT git@github-secondary
# Check which SSH key is being used
ssh-add -l
# Verify SSH config parsing
```

```
# verify SSH config parsing
ssh -F ~/.ssh/config -G github-secondary
# Check Git configuration hierarchy
git config --list --show-origin | grep user
```

Quick Reference Commands

Shell

```
# Generate SSH key
ssh-keygen -t ed25519 -C "email@example.com" -f ~/.ssh/id_ed25519_name
# Test SSH connection
ssh -T git@github-alias
# Clone with specific account
git clone git@github-alias:username/repo.git
# Set repository identity
git config user.name "Name"
git config user.email "email@example.com"
# Change remote URL
git remote set-url origin git@github-alias:username/repo.git
# Check configuration
git config --list --show-origin | grep user
ssh-add -l
git remote -v
```

Best Practices

Naming Conventions:

- Use descriptive SSH host aliases: `github-work,github-personal`
- Use consistent email patterns: `firstname.lastname@company.com`
- Name SSH keys descriptively: `id_ed25519_work,id_ed25519_personal`

Security:

- Use strong passphrases for SSH keys in production environments
- Regularly rotate SSH keys (annually)
- Keep separate SSH keys for different purposes
- Never share private keys

Backup and Recovery:

- Backup your SSH keys securely
- Document your SSH config setup
- Keep a record of which public keys are associated with which GitHub accounts

Git GitHub Authentication

Opinions expressed by DZone contributors are their own.

RELATED

From Command Lines to Intent Interfaces: Reframing Git Workflows Using Model Context Protocol

Secrets in Code: Understanding Secret Detection and Its Blind Spots

How to Push Docker Images to AWS Elastic Container Repository Using GitHub Actions

Optimizing GitHub Access Management for Enterprises: Enhancing Security, Scalability, and Continuity with Jenkins GitHub App Authentication and Load Balancing

✕